# 第6课：现代深度学习和前沿

- 李伟昌, weichangli@zju.edu.cn
- 浙江大学**海洋人工智能**课程讲义
- 版本 1, 2026年5月20日

**时长**： 45分钟
**目标**： 理解LLM、多模态模型、高效深度学习和负责任AI

---

## 6.1 大语言模型：规模与能力

### 什么使LLM"大"

- **GPT-3 (2020)**： 175B参数，在300B token上训练
- **PaLM (2022)**： 540B参数
- **GPT-4 (2023)**： 估计1T+参数（多模态）
- **Llama 3 (2024)**： 70B-405B参数，开放权重

### 训练流程

**第1步：预训练**

- 目标：下一个token预测
- 数据：互联网规模文本（网页、书籍、代码）
- 计算：数千GPU/TPU数周/月

**第2步：监督微调（SFT）**

- 在高质量指令-响应对上训练
- 使模型有帮助并遵循指令

**第3步：RLHF（来自人类反馈的强化学习）**

1. 从人类偏好训练奖励模型
2. 使用PPO针对奖励模型优化策略
3. 产生更安全、更有帮助的输出

**现代**： DPO（直接偏好优化）- 更简单，无单独奖励模型

### 涌现能力

规模上出现的能力：

- 上下文学习（少样本提示）
- 思维链推理
- 代码生成
- 无需显式训练的翻译

**缩放定律**：

$$\text{Loss} \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

其中$N$ = 参数，$D$ = 训练token

---

## 6.2 提示工程

### 什么是提示工程?

设计输入以在无需改变模型参数的情况下从LLM获得期望输出。

### 关键技术

**零样本**：

```
分类情感："This movie was amazing!"
情感：正面
```

**少样本**：

```
文本："I loved this restaurant!" → 正面
文本："Terrible service." → 负面
文本："The food was okay." →
```

**思维链**：

```
Q：Roger有5个球。他买了2罐，每罐3个。他有多少个？
A：让我们一步步思考。Roger开始有5个。他买了2罐3个的，
   所以是6个。5 + 6 = 11。答案是11。
```

**系统提示**：

```
你是一个有帮助的助手。简洁准确地回答。
用户：...
助手：...
```

### 高级技术

- **自一致性**： 生成多个答案，取多数投票
- **思维树**： 探索多个推理路径
- **检索增强生成（RAG）**： 将相关文档注入上下文

---

## 6.3 Vision Transformers

### 将Transformers应用于图像

**ViT（Vision Transformer）- Dosovitskiy等人 2020**：

1. 将图像分割为patches（例如，16×16像素）
2. 展平并线性嵌入每个patch
3. 添加位置嵌入
4. 馈送到标准Transformer编码器
5. [CLS] token用于分类

```
图像 (224×224×3)
    ↓
Patchify (16×16) → 196 patches × 768 dims
    ↓
+ 位置嵌入
    ↓
Transformer编码器 × L
    ↓
MLP头 → 类别预测
```

**性能**： 在足够数据预训练时，ViT-Large匹配或超过ResNet在ImageNet上的表现。

### 混合方法

- **Swin Transformer**： 分层、移位窗口以提高效率
- **ConvNeXt**： 受Transformers启发的现代纯CNN

---

## 6.4 多模态模型

### CLIP - 连接视觉和语言

**架构**：

- 图像编码器（ResNet或ViT）
- 文本编码器（Transformer）
- 用对比目标训练

**目标**： 最大化匹配图像-文本对的相似性，非匹配对最小化。

**应用**：

- 零样本图像分类
- 从文本检索图像
- 指导图像生成（DALL-E、Stable Diffusion）

```
from transformers import CLIPProcessor, CLIPModel

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# 计算相似性
inputs = processor(text=["a photo of a cat", "a photo of a dog"],
                   images=image, return_tensors="pt", padding=True)
outputs = model(**inputs)
logits_per_image = outputs.logits_per_image
```

### 文本到图像生成

**DALL-E 2/3**：

- CLIP文本编码
- 扩散模型用于生成
- 高质量、可控生成

**Stable Diffusion**：

- 潜在扩散（在压缩潜在空间操作）
- 开源，在消费级GPU上运行
- 催生生成式AI应用爆炸

**扩散过程**：

1. 前向：逐渐向图像添加噪声
2. 反向：模型学习去噪
3. 采样：从噪声开始，以文本为条件迭代去噪

---

## 6.5 大规模训练

### 分布式训练策略

**数据并行**：

- 每个GPU有完整模型副本
- 批次跨GPU分割
- All-reduce同步梯度

**模型并行**：

- 跨GPU分割模型层
- 每个GPU持有子集层
- 前向/后向传递在GPU间通信

**流水线并行**：

- 微批次通过模型分区流水线
- 更好的GPU利用率

**ZeRO（零冗余优化器）**：

- 分区优化器状态、梯度、参数
- 每个GPU持有1/N优化器状态
- 支持训练更大模型

**FSDP（完全分片数据并行）**：

- PyTorch原生
- 分片模型参数、梯度、优化器状态
- 类似ZeRO-3

```
# PyTorch FSDP
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP

model = FSDP(model)
```

### 混合精度训练

- 权重：FP32（主副本）
- 激活/梯度：FP16/BF16
- ~2倍加速，~2倍内存减少
- BF16优于FP16（无需缩放）

---

## 6.6 高效深度学习

### 模型压缩

**量化**：

- INT8：4倍更小，通常可忽略的精度损失
- INT4：8倍更小，更多损失但可用
- GPTQ、AWQ：LLM的后训练量化

```
# PyTorch动态量化
import torch.quantization
model_int8 = torch.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8
)
```

**剪枝**：

- 移除小幅度权重
- 结构化（移除神经元）vs 非结构化（稀疏）

**知识蒸馏**：

- 训练小"学生"模仿大"教师"
- 使用软目标（温度缩放概率）

### 高效架构

**专家混合（MoE）**：

- 每个输入只激活参数子集
- 稀疏激活支持巨大模型
- 示例：Mixtral 8×7B（8个专家，~47B有效参数，~13B激活）

**状态空间模型（Mamba）**：

- 线性时间复杂度（vs 注意力的二次）
- 选择性状态空间
- 在长序列上与Transformers竞争

---

## 6.7 伦理、安全和负责任AI

### 对LLM的担忧

**偏见**：

- 模型反映训练数据中的偏见
- 性别、种族、文化刻板印象
- 放大社会偏见

**幻觉**：

- 自信地生成虚假信息
- 没有事实核查难以检测
- 事实应用的关键问题

**滥用**：

- 生成虚假信息
- 自动化社会工程
- 恶意软件代码生成

**环境影响**：

- 训练大模型消耗大量能源
- 大规模推理有持续碳成本

### 缓解策略

**RLHF**： 训练模型拒绝有害请求

**Constitutional AI**： 用原则训练模型

**红队**： 故意测试故障模式

**水印**： 在生成文本中嵌入可检测信号

**透明度**：

- 记录能力、局限性、偏见的模型卡片
- 数据集的数据表

### 最佳实践

1. 记录已知局限性
2. 部署前测试偏见
3. 实施人工监督
4. 允许用户反馈
5. 定期安全审计

---

## 6.8 未来方向

### 活跃研究领域

**1. 长上下文**：

- 当前：128K-200K token
- 目标：数百万token
- 方法：环形注意力、循环、压缩

**2. 多模态理解**：

- 视觉、语言、音频的真正整合
- 具身AI的世界模型

**3. 推理**：

- 更好的数学推理
- 因果理解
- 规划和工具使用

**4. 效率**：

- 二次注意力的次二次替代
- 设备上部署
- 持续学习

**5. 可解释性**：

- 理解模型学习什么
- 机械可解释性
- 使模型与人类价值观对齐

### 值得关注的范式转变

- 测试时计算（o1、推理模型）
- 基于智能体的系统
- 神经符号AI
- 硬件-软件协同设计

---

## 6.9 实验：使用现代LLM

### 练习1：提示工程（20分钟）

在推理任务上比较不同提示策略：

- 零样本
- 少样本
- 思维链

在一组数学文字题上评估准确率。

### 练习2：RAG实现

构建一个简单的RAG系统：

1. 使用嵌入索引文档
2. 为查询检索相关块
3. 用检索到的上下文生成答案

```
# 简单RAG流程
from transformers import AutoTokenizer, AutoModel

# 嵌入文档
doc_embeddings = embed_documents(docs)

# 查询
query_embedding = embed(query)
relevant_docs = retrieve(query_embedding, doc_embeddings, k=3)

# 用上下文生成
context = "\n".join(relevant_docs)
prompt = f"Context: {context}\n\nQuestion: {query}\nAnswer:"
response = generate(prompt)
```

### 练习3：模型量化

比较模型大小和推理速度：

- FP16基线
- INT8量化
- 在测试集上测量困惑度

---

## 关键要点

1. LLM遵循预训练 → SFT → RLHF流程
2. 提示工程是强大的零参数技术
3. Vision Transformers将自注意力应用于图像patches
4. 多模态模型（CLIP、DALL-E）连接不同模态
5. 大规模训练需要分布式策略和混合精度
6. 效率技术（量化、蒸馏）支持部署
7. 负责任AI需要解决偏见、安全和环境问题
8. 该领域正在快速发展，有许多开放的研究方向

---

## 课程总结

### 我们涵盖的内容

| 课次 | 主题 | 关键概念 |
|---|---|---|
| 1 | 基础 | 神经元、激活、前向传播、PyTorch基础 |
| 2 | 训练 | 反向传播、SGD/Adam、正则化、批归一化 |
| 3 | CNN | 卷积、架构、迁移学习 |
| 4 | RNN | LSTM、GRU、词嵌入、seq2seq |
| 5 | Transformers | 自注意力、BERT、GPT、现代LLM |
| 6 | 前沿 | 规模、多模态、效率、负责任AI |

### 下一步

1. **实践**： 从头开始实现论文
2. **竞赛**： Kaggle比赛
3. **研究**： 阅读最近论文，识别有趣问题
4. **构建**： 端到端项目
5. **专业化**： 计算机视觉、NLP、RL、系统等

### 持续学习资源

**研究和实现**

- Papers with Code - 最新论文+实现
- Hugging Face - 模型、数据集、spaces
- Distill.pub - 高质量ML解释
- MLSys - 机器学习系统

**Andrej Karpathy资源（致谢）**

本课程的视觉教学方法——特别是计算图、逐步反向传播示例和直观解释——大量借鉴了**Andrej Karpathy**的教育材料：

- **Neural Networks: Zero to Hero** - 从头构建神经网络的视频系列
- **micrograd** - ~100行autograd引擎演示反向传播
- **Karpathy's Blog** - 精华阅读包括"A Recipe for Training Neural Networks"

**逐步教程**

- **HMKCODE's Backpropagation Step by Step** - 第2课中的具体数值演练和视觉梯度流图受此教程启发，提供了详细的逐步计算和实际数字

---

## 谢谢！

有问题？联系：[weichangli@zju.edu.cn]